

Modular Microservices Architecture Document

Prepared for: Third-Party Development Company

Prepared by: [Your Company Name]

Date: April 10, 2026

1. Project Objective

The objective of this project is to build a **modular enterprise software platform based on microservices architecture**.

The platform must support **independent module-based development, deployment, and licensing**.

For example: - CRM Module only - HRMS Module only - Billing Module only - Inventory Module only - Complete ERP Suite

Each module must work as: - **standalone product** - **integrated product within the full ecosystem**

This architecture must support scalability, maintainability, and independent deployments.

2. Architecture Approach

The complete solution must follow **Microservices + Modular Monolith Frontend Architecture**.

High-Level Layers

1. Frontend Layer
 2. API Gateway Layer
 3. Microservices Layer
 4. Shared Platform Services
 5. Database Layer
 6. DevOps / Deployment Layer
-

3. Frontend Architecture Structure

Technology Stack Updated: Angular + .NET Core 9 + Microsoft SQL Server Frontend must be **module-based and scalable**.

Recommended Frontend Stack

- Angular 18+
- TypeScript
- Angular Material / Tailwind CSS
- NgRx / Signals
- Module Federation (optional for micro-frontend support)

Frontend Folder Structure

```
frontend/  
|  
├─ apps/  
|   ├─ crm/  
|   ├─ hrms/  
|   ├─ billing/  
|   └─ inventory/  
|  
├─ shared/  
|   ├─ components/  
|   ├─ hooks/  
|   ├─ services/  
|   ├─ utils/  
|   └─ auth/  
|  
├─ layouts/  
├─ routes/  
└─ theme/
```

Frontend Requirements

- Each module should have independent routes
- Shared UI components must be reusable
- Common authentication flow
- Role-based menu rendering
- Feature-based lazy loading
- Independent module enable/disable support

Example: If client purchases only CRM module, frontend should render only CRM routes.

4. Backend Architecture (Microservices)

Backend must be **fully microservice-driven**.

Each business domain must be a separate service.

Example Services

```
backend/  
|  
├─ api-gateway/  
├─ auth-service/  
├─ user-service/  
├─ crm-service/  
├─ hrms-service/  
├─ billing-service/  
├─ inventory-service/  
├─ notification-service/  
├─ reporting-service/  
└─ file-service/
```

Rules

Each service must contain: - separate codebase - separate deployment pipeline - separate database/schema
- separate migrations - separate logging - separate monitoring

5. Suggested Backend Tech Stack

Primary Backend Framework: .NET Core 9 Recommended: - .NET Core 9 (ASP.NET Core Web API / Minimal APIs) - REST + optional GraphQL - gRPC for internal communication - RabbitMQ / Kafka for async communication - Redis for caching

6. API Gateway Architecture

All frontend traffic must pass through API Gateway.

Responsibilities

- authentication validation
- request routing
- rate limiting
- request logging
- module-based routing
- tenant-based routing

Example:

```
/api/crm → CRM Service  
/api/hrms → HRMS Service  
/api/billing → Billing Service
```

7. Authentication & Authorization

Must be centralized.

Recommended

- OAuth2 / JWT
- SSO ready
- RBAC (Role Based Access Control)
- Permission-based access

Example Roles

- Super Admin
- Company Admin
- Manager
- Employee
- Sales User

8. Database Architecture

Every module must have **independent database ownership**.

Recommended Approach

Database per service

Example:

```
crm_db  
hrms_db
```

```
billing_db
inventory_db
```

This ensures: - independent scaling - no tight coupling - easy deployment - isolated failures

Recommended DB

- Microsoft SQL Server (primary database)
- Optional MongoDB for document-heavy or audit-log modules

9. Database Migration Structure

Migration must be maintained separately for every service.

```
crm-service/
├─ migrations/
│   └─ V1__create_leads_table.sql
│   └─ V2__create_opportunities.sql
```

Recommended Tools

- Entity Framework Core 9 Migrations
- FluentMigrator (optional enterprise standard)
- SQL Server DACPAC deployment (optional DevOps flow)

Every deployment must automatically run migrations.

10. Inter-Service Communication

Use hybrid communication.

Synchronous

- REST
- gRPC

Asynchronous

- Kafka / RabbitMQ

Example events: - customer_created - invoice_generated - employee_onboarded

This is mandatory for scalable architecture.

11. Module-Based Deployment Requirement

This is one of the most important requirements.

Each module must be deployable independently.

Example: - Client A → only CRM - Client B → CRM + Billing - Client C → Full Suite

Deployment Design

Every service must have separate Docker image.

```
crm-service:v1  
billing-service:v1  
hrms-service:v1
```

This allows module-based deployment.

12. Containerization

All services must be containerized.

Mandatory

- Docker
 - Docker Compose for local
 - Kubernetes for production
-

13. Kubernetes Deployment Structure

```
k8s/  
├─ crm/  
│   ├── deployment.yaml  
│   └── service.yaml
```

```
|  
|— billing/  
|— hrms/
```

Each module must have: - Deployment - Service - ConfigMap - Secret - HPA - Ingress

14. CI/CD Pipeline

Each microservice must have separate CI/CD.

Pipeline Flow

```
Code Push  
→ Build  
→ Unit Test  
→ Security Scan  
→ Docker Build  
→ Push Registry  
→ Deploy Kubernetes  
→ Run Migrations  
→ Smoke Test
```

Recommended Tools

- GitHub Actions
 - GitLab CI/CD
 - Jenkins
 - ArgoCD
-

15. Logging & Monitoring

Mandatory observability stack.

Logging

- ELK Stack
- Loki + Grafana

Monitoring

- Prometheus
- Grafana

Tracing

- OpenTelemetry
 - Jaeger
-

16. Multi-Tenant Architecture

Platform should support multiple clients.

Options

- Shared DB + tenant_id
- Separate DB per client

Recommended: **tenant-isolated schema/database model**

17. Security Requirements

Mandatory: - JWT validation - HTTPS only - API encryption - secret vault - audit logs - OWASP standards - input validation - SQL injection prevention

18. Development Standards

Mandatory coding standards: - clean architecture - DDD approach - SOLID principles - unit tests - integration tests - API versioning - swagger docs

19. Deliverables Expected from Third Party

Third-party company must provide:

1. HLD (High-Level Design)
2. LLD (Low-Level Design)
3. API documentation

4. DB schema design
 5. migration strategy
 6. deployment architecture
 7. Kubernetes manifests
 8. CI/CD pipelines
 9. test strategy
 10. security design
-

20. Final Key Requirement

Architecture must be designed in such a way that:

Any module can be sold, deployed, upgraded, and maintained independently without impacting other modules.

This is the primary business requirement.